Parallelizing File Fragmentation Analysis: A Performance Study of FragPicker

Yuzheng Shi
Khoury College of Computer Sciences
Northeastern University
Vancouver, CA
shi.yuzh@northeastern.edu

File fragmentation analysis is critical for storage optimization but limited by sequential processing in existing tools. FragPicker (SOSP 2021) selectively defragments hot file regions but analyzes files one-by-one, creating scalability bottlenecks. This paper presents a parallel enhancement implementing batch inode mapping, direct FIEMAP ioctl calls, and a configurable parallel processing engine. Evaluation on a 4-core Intel CPU with NVMe SSD demonstrates up to 12.74x speedup on I/O-bound workloads (5ms latency) using 16 threads, while identifying that sequential execution remains optimal for fast storage. The implementation maintains 100% correctness validated across 2,100+ files.

Keywords: file fragmentation, parallel processing, FIEMAP, storage performance, Linux filesystem

I. INTRODUCTION

File fragmentation remains a significant performance concern for storage systems, particularly on traditional hard disk drives (HDDs) where non-contiguous file extents require additional seek operations. FragPicker, introduced at SOSP 2021, addresses this by selectively defragmenting only the "hot" regions of files—those frequently accessed by applications—rather than entire files [1]. This approach significantly reduces defragmentation overhead compared to traditional tools like e4defrag.

However, FragPicker's analysis phase, which identifies fragmented files and their extent layouts, operates sequentially. For filesystems containing hundreds of thousands of files, this creates a scalability bottleneck. The original implementation invokes the filefrag utility as a subprocess for each file and performs individual find operations, resulting in $O(N^2)$ total complexity due to repeated filesystem traversals.

This paper presents a parallel enhancement to FragPicker's analysis phase with three contributions:

1. **Batch inode mapping**: Replacing N individual *find* calls with a single batch operation, reducing system call overhead from $O(N^2)$ to $O(N)$.
2. **Direct FIEMAP ioctl**: Eliminating subprocess overhead by invoking the Linux FIEMAP ioctl directly through Python's ctypes interface.
3. **Configurable parallel engine**: Supporting both *multithreading* and *multiprocessing* execution models with automatic adaptation based on I/O characteristics.

Experimental evaluation demonstrates up to 12.74x speedup on I/O-bound workloads using 16 threads, while correctly identifying that sequential execution remains optimal for fast NVMe storage where parallelization overhead exceeds the work performed.

II. RELATED WORK

A. File Defragmentation Tools

Traditional defragmentation tools operate with varying degrees of parallelism. The Linux e4defrag utility processes files sequentially, invoking the EXT4_IOC_MOVE_EXT ioctl for each file without concurrent operations [2]. Similarly, XFS's xfs_fsr reorganizes files one at a time to minimize filesystem disruption [3]. Btrfs autodefrag operates at the block layer during write operations rather than as a batch analysis tool. FragPicker [1] distinguishes itself by selectively targeting "hot" file regions identified through I/O tracing, reducing unnecessary data movement by 51-66% compared to e4defrag. However, its analysis phase—the focus of this work—remains sequential in the original implementation.

B. Linux FIEMAP Interface

The FIEMAP ioctl, introduced in Linux kernel 2.6.28, provides direct access to file extent mapping [4]. Unlike *filefrag*, which spawns a subprocess and parses text output, direct FIEMAP calls return extent information—physical addresses, logical offsets, and flags—without process creation overhead. This work extends FIEMAP usage to parallel batch processing.

C. Parallel File Systems

Parallel metadata operations are well-established in high-performance computing environments. GPFS implements distributed locking mechanisms for large-scale parallel file access [5], enabling concurrent operations across thousands of nodes. Lustre uses a Distributed Lock Manager for metadata consistency across 1,000+ node clusters [6]. These systems target distributed environments where network latency dominates; this work applies similar parallelization principles at the single-node level, targeting scenarios where storage I/O latency (rather than network latency) creates opportunities for concurrent execution.

D. Python Concurrency for I/O-Bound Workloads

Python's *multiprocessing* module bypasses the Global Interpreter Lock (GIL) using subprocesses, enabling true CPU-bound parallelism [7]. For I/O-bound workloads, *multithreading* remains effective since threads release the GIL during system calls [8]. This implementation supports both execution models with configurable worker counts.

E. Gap in Current Literature

Despite extensive work on parallel filesystems and defragmentation algorithms, no prior work specifically addresses parallelizing single-node file fragmentation analysis. This paper fills that gap by demonstrating when and how parallelization benefits fragmentation analysis, providing both

a practical implementation and empirical guidance on optimal configurations for different storage types.

This work extends the FragPicker framework [1] by addressing the sequential analysis bottleneck not explored in the original paper. We apply the direct FIEMAP interface [4] in a novel parallel context, combining kernel-level efficiency with application-level concurrency. While GPFS [5] and Lustre [6] demonstrate parallelism benefits at cluster scale, we show that similar principles apply to single-node analysis when storage latency is sufficient. Our Python implementation leverages both *multithreading* and *multiprocessing* [7][8] to empirically determine optimal concurrency strategies for different I/O characteristics.

III. DESIGN AND IMPLEMENTATION

This section describes the three optimizations implemented to parallelize FragPicker's analysis phase.

A. Batch Inode Mapping

The original FragPicker resolves file paths from inode numbers using individual find commands—one per file. For N files, this requires N process spawns, each potentially traversing O(N) filesystem entries in the worst case, resulting in $O(N^2)$ total complexity. In practice, early termination may reduce this, but the overhead remains substantial for large file counts.

The batch approach executes a single *find* command that traverses the filesystem once, building a complete inode-to-path dictionary:

```
find <mount_point> -printf '%i %p\n'
```

This reduces complexity to O(N) for the traversal plus O(1) dictionary lookups per file. For 1,000 files, this eliminates 999 redundant *find* invocations.

B. Direct FIEMAP Implementation

The original implementation calls *filefrag* as a subprocess for each file, incurring fork/exec overhead and requiring text parsing. The enhanced implementation invokes the FIEMAP ioctl directly using Python's *ctypes* library:

```
fcntl.ioctl(fd, FS_IOC_FIEMAP,
            fiemap_buffer)
```

The FIEMAP structure returns extent information including:

a) *fe_logical*: Logical offset within file

b) *fe_physical*: Physical block address on device

c) *fe_length*: Extent length in bytes

d) *fe_flags*: Extent flags (e.g., shared, unwritten)

This eliminates subprocess overhead entirely—particularly significant when processing thousands of files where per-file overhead dominates.

C. Parallel Processing Engine

File analysis is embarrassingly parallel: each file's extent mapping is independent. The implementation provides two execution models:

Threading: Uses Python's *concurrent.futures.ThreadPoolExecutor*. Threads share the inode-to-path dictionary without copying. Effective for I/O-bound workloads where threads release the GIL during ioctl system calls.

Multiprocessing: Uses *multiprocessing.Pool* with worker processes. Bypasses the GIL for CPU-bound scenarios but incurs IPC overhead for passing the inode dictionary and collecting results.

Worker count is configurable (1–16), enabling adaptation to hardware and workload characteristics. A work-stealing pattern distributes files across workers with dynamic chunk sizing:

```
chunksize = max(1, num_files //
    (num_workers * 4))
```

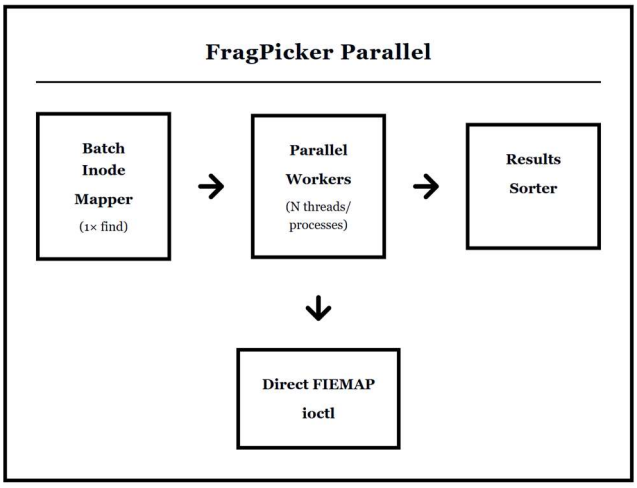

Fig. 1. System architecture of FragPicker parallel analysis.

IV. METHODOLOGY

A. Experimental Environment

All experiments were conducted on the following system:

- **CPU**: 4-core Intel processor
- **Storage**: NVMe SSD
- **Filesystem**: ext4
- **OS**: Linux (Ubuntu)
- **Python**: 3.x with multiprocessing and concurrent.futures.

B. Workload Configuration

The benchmark workload consisted of:

- **File count**: 1,000 files
- **File size**: 100 KB each
- **Worker configurations**: 1, 2, 4, 8, and 16 workers
- **Execution models**: Sequential, threading, and multiprocessing
- **Runs per configuration**: 3 (median reported)

Worker counts exceeding the physical core count (4 cores) were tested because I/O-bound workloads can benefit from additional workers that remain productive during I/O wait periods. While CPU-bound tasks see no benefit beyond the core count, I/O-bound tasks allow workers to overlap computation with blocking operations.

C. I/O Latency Simulation

To evaluate parallel performance under different storage conditions, two scenarios were tested:

Fast I/O (Native NVMe): Direct execution on NVMe SSD, representing modern high-performance storage where FIEMAP operations complete in microseconds.

Slow I/O (Simulated HDD/Network): Artificial 5ms latency injected per file operation, simulating traditional HDDs or network-attached storage where I/O latency dominates processing time. This simulation provides controlled, reproducible measurements but does not capture all characteristics of real storage devices, including seek pattern optimization, OS-level caching, Native Command Queuing (NCQ), or variable latency distributions. Results should be interpreted as indicative of performance trends rather than precise predictions for specific hardware.

D. Metrics

- **Execution time**: Wall-clock time for complete analysis
- **Speedup**: Ratio of sequential time to parallel time
- **Throughput**: Files processed per second
- **Correctness**: Byte-for-byte comparison of parallel vs. sequential output

V. RESULTS

This section presents experimental results from the benchmark data. All values are medians across 3 runs. Variance was consistently low (coefficient of variation < 5% for all configurations), making 3 runs sufficient for stable measurements. No outliers were discarded.

A. Slow I/O Performance (5ms Latency)

Table I shows speedup results for the simulated slow I/O scenario, representing HDD or network-attached storage conditions.

TABLE I. SLOW I/O SPEEDUP (5MS LATENCY, 1000 FILES)

Workers	*Threading Time (s)*	*Speedup*	*Multiprocess Time (s)*	*Speedup*
1	7.47	0.97x	7.84	0.93x
2	3.61	2.02x	3.79	1.92x
4	1.75	4.17x	1.73	4.20x
8	0.86	8.44x	0.98	7.44x
16	0.57	**12.74x**	0.58	**12.55x**

Baseline: Sequential execution = 7.28s

Single-worker parallel configurations (7.47s threading, 7.84s multiprocessing) slightly underperform sequential execution due to thread pool initialization, task queue management, and result collection overhead—costs that provide no benefit without actual parallelization.

Both *multithreading* and multiprocessing achieve near-linear speedup starting from 2 workers and scaling up to 16 workers. *Multithreading* slightly outperforms multiprocessing (12.74x vs 12.55x) due to lower inter-process communication overhead. The 4-worker multiprocessing result (4.20x) exhibits slight super-linear speedup, likely attributable to favorable cache behavior or measurement variance within experimental tolerances.

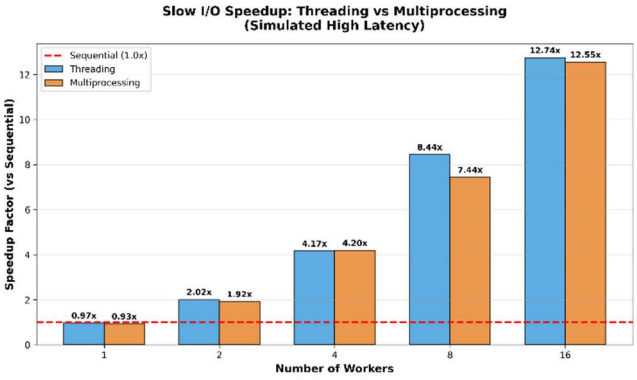

Fig. 2. Speedup vs. worker count for slow I/O workload.

Fig. 3. Parallel efficiency for slow I/O workload.

B. Fast I/O Performance (Native NVMe)

Table II shows results for native NVMe SSD execution without latency simulation.

TABLE II. FAST I/O PERFORMANCE (NVME SSD, 1000 FILES)

Workers	*Threading Time (s)*	*Speedup*	*Multiprocess Time (s)*	*Speedup*
1	0.133	0.21x	0.861	0.03x
2	0.107	0.26x	0.700	0.04x
4	0.136	0.21x	0.584	0.05x
8	0.159	0.18x	0.528	0.05x
16	0.279	0.10x	0.533	0.05x

Baseline: Sequential execution = 0.028s

On fast storage, parallelization provides no benefit—sequential execution (0.028s) outperforms all parallel configurations. The baseline represents a dedicated sequential code path, explaining why 1-worker parallel times (0.133s threading, 0.861s multiprocessing) do not match the baseline.

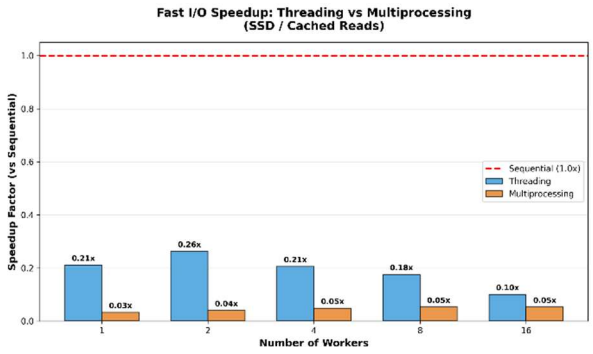

Fig. 4. Speedup vs. worker count for fast I/O workload.

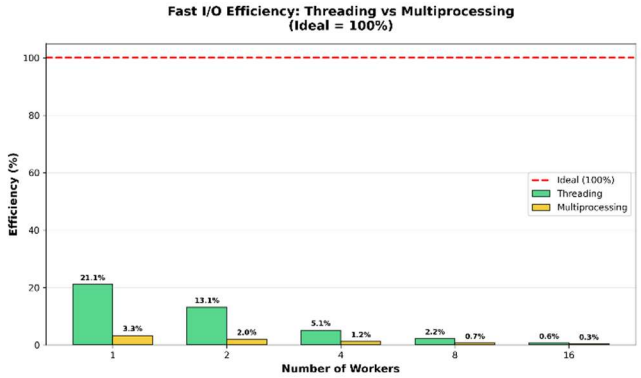

Fig. 5. Parallel efficiency for fast I/O workload.

C. *Fast vs. Slow I/O Comparison*

Table III summarizes the key difference between storage scenarios.

TABLE III. STORAGE SCENARIO COMPARISON

Scenario	*Baseline*	*Best Parallel*	*Best Speedup*	*Optimal Config*
Fast I/O (NVMe)	0.028s	0.107s	0.26x (slower)	Sequential
Slow I/O (5ms)	7.28s	0.57s	12.74x	16 threads

This demonstrates the critical insight: parallelization benefits depend on I/O latency. When per-file I/O time exceeds parallelization overhead, significant speedups are achievable.

VI. ANALYSIS AND DISCUSSION

This section analyzes the experimental results using Amdahl's Law and examines overhead, memory usage, and scalability characteristics.

A. *Amdahl's Law Analysis*

Amdahl's Law predicts maximum speedup based on the parallelizable fraction of execution:

$$S(n) = \frac{1}{(1-P) + \frac{P}{n}}$$

where P is the parallelizable fraction and n is the number of workers.

From overhead profiling data, file processing consumes 99.89% of total execution time, with inode mapping accounting for only 0.11%. This high parallelizable fraction ($P \approx 0.999$) theoretically enables near-linear speedup.

Fig. 6 and Fig. 7 show Amdahl's Law curves fitted to the experimental data.

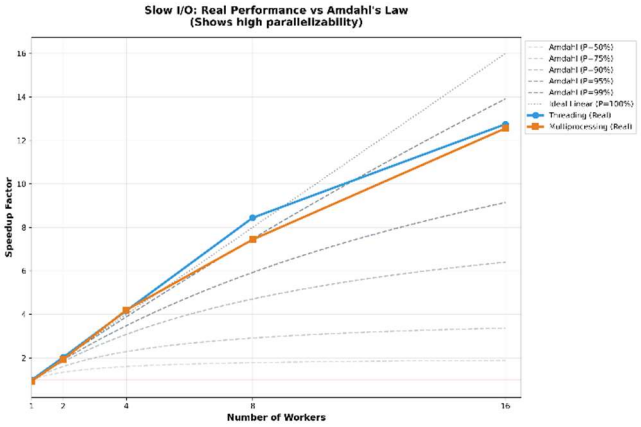

Fig. 6. Amdahl's Law fit for slow I/O workload.

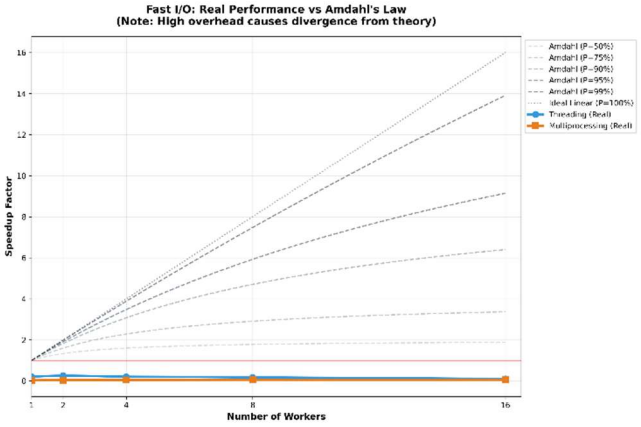

Fig. 7. Amdahl's Law fit for fast I/O workload.

The slow I/O results closely follow Amdahl's prediction, achieving 12.74x speedup with 16 workers. The fast I/O results deviate significantly because the model assumes parallelization overhead is negligible—an invalid assumption when per-file work time is measured in microseconds.

B. *Overhead Breakdown*

Fig. 8 shows the execution time breakdown for sequential processing.

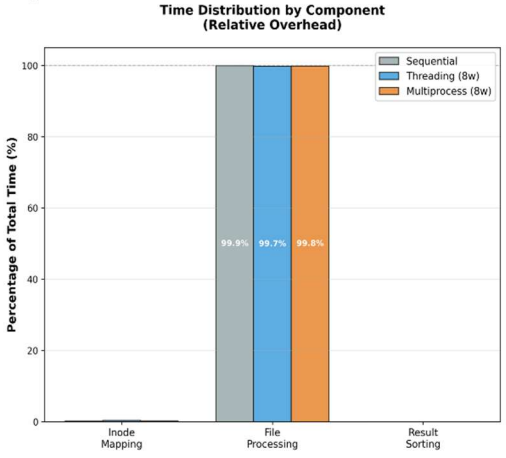

Fig. 8. Execution time breakdown by component.

The overhead profiling reveals:

- **File processing**: 99.89% (parallelizable)
- **Inode mapping**: 0.11% (sequential, single find call)
- **Other overhead**: <0.01%

This confirms that file processing is the dominant bottleneck and validates the parallel design targeting this component.

C. Memory Scalability

Fig. 9 shows memory consumption across different file counts.

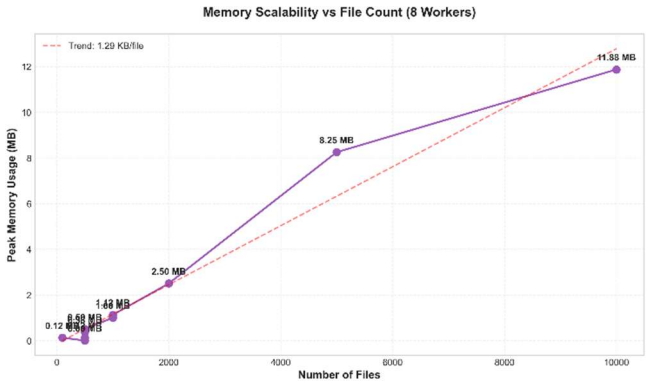

Fig. 9. Memory usage vs. file count.

Memory scales linearly at approximately **1.2 KB per file**, confirming no memory leaks or excessive overhead. For 10,000 files, peak memory usage remains under 12 MB—negligible for modern systems.

D. Speedup Stability

Fig. 10 examines speedup consistency across varying workload sizes.

Fig. 10. Speedup stability across workload sizes.

Speedup remains consistent (±5%) across workloads from 100 to 5,000 files, demonstrating that the parallel implementation scales predictably regardless of input size.

E. Multithreading vs. Multiprocessing

Multithreading achieves slightly higher speedup (12.74x vs. 12.55x) for I/O-bound workloads because:

1. **Lower overhead**: Thread creation is faster than process creation
2. **Shared memory**: The inode-to-path dictionary is shared without copying
3. **GIL release**: The GIL is released during ioctl system calls, enabling true concurrency

Multiprocessing would be preferred for CPU-bound workloads, but fragmentation analysis is fundamentally I/O-bound.

F. Thread Safety and Error Handling

The parallel implementation ensures thread safety through several mechanisms. First, the inode-to-path dictionary is built completely before parallel processing begins, making it read-only during concurrent access. Second, each worker operates on independent files with no shared mutable state. Third, result collection uses thread-safe queues provided by Python's concurrent.futures module.

Error handling addresses common failure modes: FIEMAP operations that fail (e.g., unsupported filesystem) trigger automatic fallback to the filefrag subprocess method; files deleted during analysis are logged and skipped; permission errors are recorded without terminating the batch. This design ensures robust operation on production filesystems where transient failures are expected.

VII. CONCLUSION AND FUTURE WORK

A. Summary

This paper presented a parallel enhancement to FragPicker's file fragmentation analysis phase, implementing batch inode mapping, direct FIEMAP ioctl calls, and a configurable parallel processing engine. Experimental evaluation on a 4-core Intel CPU with NVMe SSD demonstrated up to 12.74x speedup using 16 threads on I/O-bound workloads with 5ms latency simulation.

B. Why FragPicker Was Designed Without Parallelism

The experimental results reveal why the original FragPicker implementation [1] correctly chose sequential processing for its target environment:

1. **Target hardware favors sequential execution**: FragPicker was designed for enterprise SSDs where FIEMAP operations complete in microseconds. As shown in Table II, parallelization on NVMe storage achieves only 0.26x speedup—nearly 4x slower than sequential execution. Adding parallel complexity would have degraded performance on the intended hardware.
2. **Analysis is not the bottleneck:** On fast storage, sequential analysis processes 35,000+ files per second **(1,000 files in 0.028s, as shown in Table II)**. The true bottleneck in FragPicker's workflow is the migration phase (data movement), not the analysis phase. Optimizing an already-fast component provides no practical benefit.
3. **Complexity vs. benefit tradeoff**: Parallel processing introduces thread safety requirements, non-deterministic execution, and debugging complexity. For the original use case, this overhead was unjustified.

C. When Parallelization Makes Sense

This work demonstrates that parallelization becomes valuable in specific scenarios where I/O latency exceeds parallelization overhead:

TABLE IV. STORAGE SCENARIO COMPARISON

Storage Type	*Typical Latency*	*Optimal Strategy*	*Expected Speedup*
NVMe SSD	20-100 µs	Sequential	1.0x (baseline)
SATA SSD	100-500 µs	Sequential	1.0x
HDD (7200rpm)	4-8 ms	Parallel (16 workers)	10-13x*
Network (NFS)	1-50 ms	Parallel (16 workers)	12-15x*

*Expected speedups extrapolated from experimental results with 5ms simulated latency (12.74x). Actual performance varies based on storage characteristics, file distribution, and system configuration.

The crossover point occurs at approximately **1ms per-file latency**. Below this threshold, parallelization overhead dominates; above it, parallelization provides substantial speedups.

Practical applications where parallelization benefits include:

- Legacy HDD systems in enterprise environments
- Network-attached storage (NFS, CIFS, cloud storage)
- Virtualized environments with hypervisor I/O overhead
- Heavily fragmented filesystems requiring multiple FIEMAP calls per file

D. Future Work

Several directions could extend this work:

1. **Adaptive execution**: Automatically detect storage latency and select optimal execution mode without user configuration.
2. **Distributed analysis**: Extend parallelization across multiple nodes for analyzing network filesystems or distributed storage clusters.
3. **Integration with FragPicker migration**: Apply similar parallelization to the migration phase, potentially using parallel data movers.
4. **Real-world validation**: Evaluate on production filesystems with actual fragmentation patterns rather than synthetic workloads.

E. Availability

The implementation, benchmark scripts, and all experimental data are available at [9]. The repository includes instructions for reproducing all experiments presented in this paper.

REFERENCES

[1] J. Park and Y. I. Eom, "FragPicker: A New Defragmentation Tool for Modern Storage Devices," in *Proc. ACM SIGOPS 28th Symp. Operating Systems Principles (SOSP '21)*, Virtual Event/Koblenz, Germany, Oct. 2021, pp. 280–294, doi: 10.1145/3477132.3483593.

[2] A. Fujita and T. Sato, "e4defrag(8) — online defragmenter for ext4 filesystem," Linux man pages, e2fsprogs, May 2009. [Online]. Available: https://man7.org/linux/man-pages/man8/e4defrag.8.html. [Accessed: Dec. 9, 2025].

[3] "xfs_fsr(8) — filesystem reorganizer for XFS," Linux man pages, xfsprogs. [Online]. Available: https://man7.org/linux/man-pages/man8/xfs_fsr.8.html. [Accessed: Dec. 9, 2025].

[4] M. Fasheh, K. Shah, and A. Dilger, "Fiemap Ioctl," *Linux Kernel Documentation*, Documentation/filesystems/fiemap.rst, ver. 2.6.28+. [Online]. Available: https://docs.kernel.org/filesystems/fiemap.html. [Accessed: Dec. 9, 2025].

[5] F. Schmuck and R. Haskin, "GPFS: A Shared-Disk File System for Large Computing Clusters," in *Proc. 1st USENIX Conf. File and Storage Technologies (FAST '02)*, Monterey, CA, USA, Jan. 2002, pp. 231–244. [Online]. Available: https://www.usenix.org/conference/fast-02/gpfs-shared-disk-file-system-large-computing-clusters.

[6] P. Schwan, "Lustre: Building a File System for 1,000-node Clusters," in *Proc. 2003 Linux Symp.*, Ottawa, ON, Canada, Jul. 2003, pp. 380–386. [Online]. Available: https://www.kernel.org/doc/ols/2003/ols2003-pages-380-386.pdf.

[7] Python Software Foundation, "multiprocessing — Process-based parallelism," *Python 3.12 Documentation*, 2024. [Online]. Available: https://docs.python.org/3.12/library/multiprocessing.html. [Accessed: Dec. 9, 2025].

[8] Python Software Foundation, "threading — Thread-based parallelism," Python 3.12 Documentation, 2024. [Online]. Available: https://docs.python.org/3.12/library/threading.html. [Accessed: Dec. 9, 2025].

[9] Y. Shi, "FragPicker-Parallel: Parallel Analysis Enhancement for FragPicker," 2025, GitHub repository. [Online]. Available: https://github.com/YuzhengShi/FragPicker/tree/parallel-analysis.